

Dashboard Deployment Process

Step 1: Code Refactoring

After the initial code for the dashboard was developed, we refactored the code to enhance modularity. This involved organizing the code into a more structured format by creating a separate folder for the pages and splitting these pages into individual Python files.

Step 2: Code Formatting and Linting

To maintain a high standard of code quality, we utilized several tools for code formatting and linting. The tools used were:

- **Isort:** For sorting imports
- **Black:** For code formatting
- **Flake8:** For linting

Formatting and Linting Process

1. **Isort on Page Files:** We started by running Isort on the page files to ensure that all imports were sorted correctly (see Figure 1).

```
# poetry run isort heatmap.py
Fixing /home/yld/2023-24d-fail-adsai-teamwork-t2/ILO_7_Deployment/project/project/pages/heatmap.py
# poetry run isort information.py
poFixing /home/yld/2023-24d-fail-adsai-teamwork-t2/ILO_7_Deployment/project/project/pages/information.py

# poetry run isort prediction.py
Fixing /home/yld/2023-24d-fail-adsai-teamwork-t2/ILO_7_Deployment/project/project/pages/prediction.py
# poetry run isort test.py
```

Figure 1. Running Isort on page files.

2. **Black on Page Files:** Next, we applied Black to the page files for automatic code formatting (see Figure 2).

```
# poetry run black heatmap.py
reformatted heatmap.py

All done! 🍌🍰🍌
1 file reformatted.
# poetry run black information.py
reformatted information.py

All done! 🍌🍰🍌
1 file reformatted.
# poetry run black prediction.py
reformatted prediction.py

All done! 🍌🍰🍌
1 file reformatted.
```

Figure 2. Running Black on page files.

3. **Isort and Black on Other Python Files:** After formatting the page files, we proceeded to run both Isort and Black on the remaining Python files to maintain consistency across the codebase (see Figure 3).

```
# poetry run black data_cleaning.py
reformatted data_cleaning.py

All done! 🍌🍰🍌
1 file reformatted.
# poetry run black model.py
reformatted model.py

All done! 🍌🍰🍌
1 file reformatted.
# poetry run black app.py
reformatted app.py

All done! 🍌🍰🍌
1 file reformatted.
# poetry run isort data_cleaning.py
Fixing /home/yld/2023-24d-fai1-adsai-teamwork-t2/ILO_7_Deployment/project/project/data_cleaning.py
# poetry run isort model.py
Fixing /home/yld/2023-24d-fai1-adsai-teamwork-t2/ILO_7_Deployment/project/project/model.py
# poetry run isort app.py
Fixing /home/yld/2023-24d-fai1-adsai-teamwork-t2/ILO_7_Deployment/project/project/app.py
# poetry run flake8 data_cleaning.py
```

Figure 3. Running Isort and Black on other Python files.

4. **Error Fixing with Black:** Initially, we ran Isort followed by Black, which fixed a significant number of formatting errors, as illustrated below (see Figure 4).

```
# poetry run flake8 data_cleaning.py
data_cleaning.py:20:80: E501 line too long (150 > 79 characters)
data_cleaning.py:27:1: E302 expected 2 blank lines, found 1
data_cleaning.py:39:62: W291 trailing whitespace
data_cleaning.py:41:1: W293 blank line contains whitespace
data_cleaning.py:44:1: W293 blank line contains whitespace
data_cleaning.py:47:1: W293 blank line contains whitespace
data_cleaning.py:50:1: W293 blank line contains whitespace
data_cleaning.py:53:80: E501 line too long (128 > 79 characters)
data_cleaning.py:54:80: E501 line too long (97 > 79 characters)
data_cleaning.py:55:1: W293 blank line contains whitespace
data_cleaning.py:64:1: W293 blank line contains whitespace
data_cleaning.py:65:80: E501 line too long (91 > 79 characters)
data_cleaning.py:66:1: W293 blank line contains whitespace
data_cleaning.py:69:80: E501 line too long (80 > 79 characters)
data_cleaning.py:71:1: W293 blank line contains whitespace
data_cleaning.py:74:1: E302 expected 2 blank lines, found 1
data_cleaning.py:93:80: E501 line too long (82 > 79 characters)
data_cleaning.py:99:1: E302 expected 2 blank lines, found 1
data_cleaning.py:107:80: E501 line too long (97 > 79 characters)
data_cleaning.py:116:1: E302 expected 2 blank lines, found 1
# poetry run black data_cleaning.py
reformatted data_cleaning.py

All done! 🍏🍰🍏
1 file reformatted.
# poetry run flake8 data_cleaning.py
data_cleaning.py:20:80: E501 line too long (150 > 79 characters)
data_cleaning.py:100:80: E501 line too long (80 > 79 characters)
data_cleaning.py:125:80: E501 line too long (82 > 79 characters)
data_cleaning.py:141:80: E501 line too long (81 > 79 characters)
```

Figure 4. Running Isort followed by Black to fix errors.

Step 3: Adding Docstrings

After the code was formatted, we created docstrings to provide documentation and information about specific functions. This step was crucial for enhancing code readability and maintainability.

Step 4: Implementing Logging

Next, we added logging to all Python scripts. This addition was essential for tracking the execution of the code, debugging, and maintaining a log of operations for future reference.

Step 5: Generating Sphinx Documentation

To generate comprehensive documentation, we used Sphinx. This step involved configuring Sphinx and generating HTML documentation from the docstrings and additional documentation files (See Figure 5)

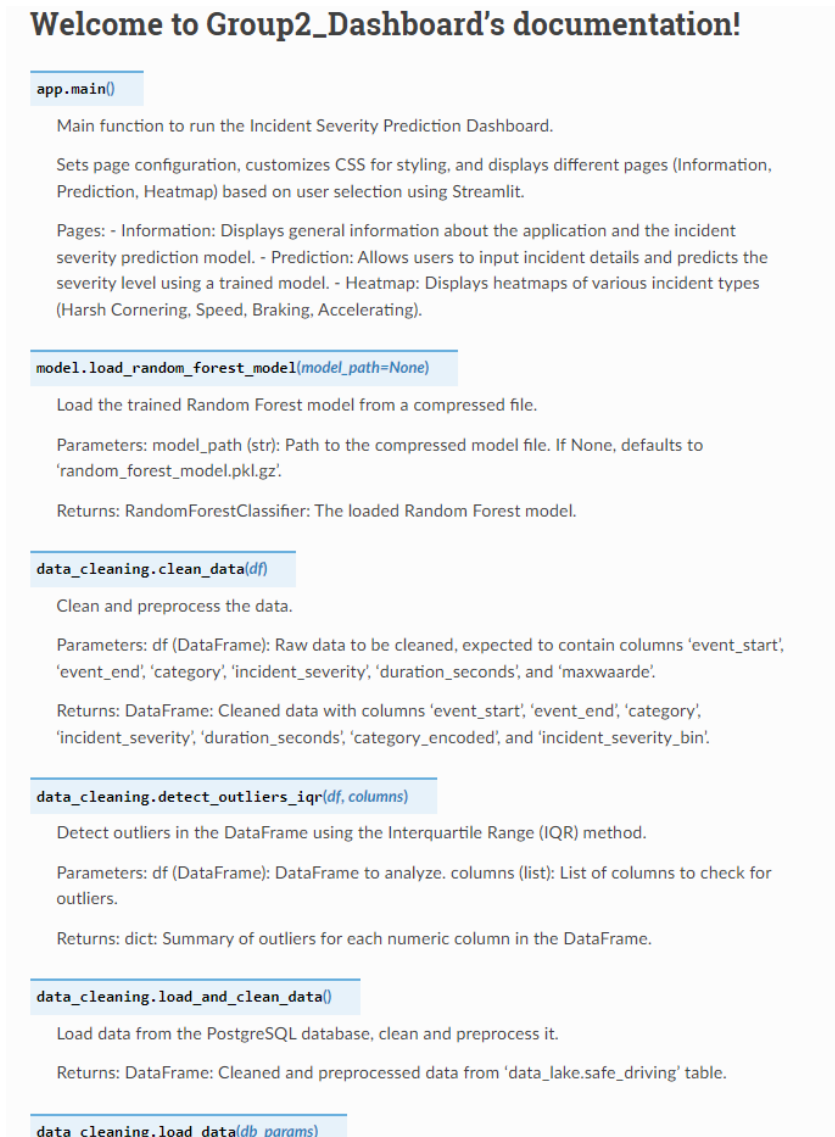


Figure 5. Screenshot of the Sphinx html index

Evidence:

https://github.com/BredaUniversityADSAI/2023-24d-fai1-adsai-teamwork-t2/blob/059b6e07397000ce9f32b25eb4ebfae9fff4d4dd/ILO_7_Deployment/project/docs/build/html/index.html

Step 6: Unit Testing

Finally, we performed unit testing to ensure that our code was functioning correctly. We used unit tests to validate the behavior of individual units of code.

Unit Testing Process

1. **Coverage Report on Files:** We generated a coverage report to verify the extent of our tests across the different files (see Figure 6).

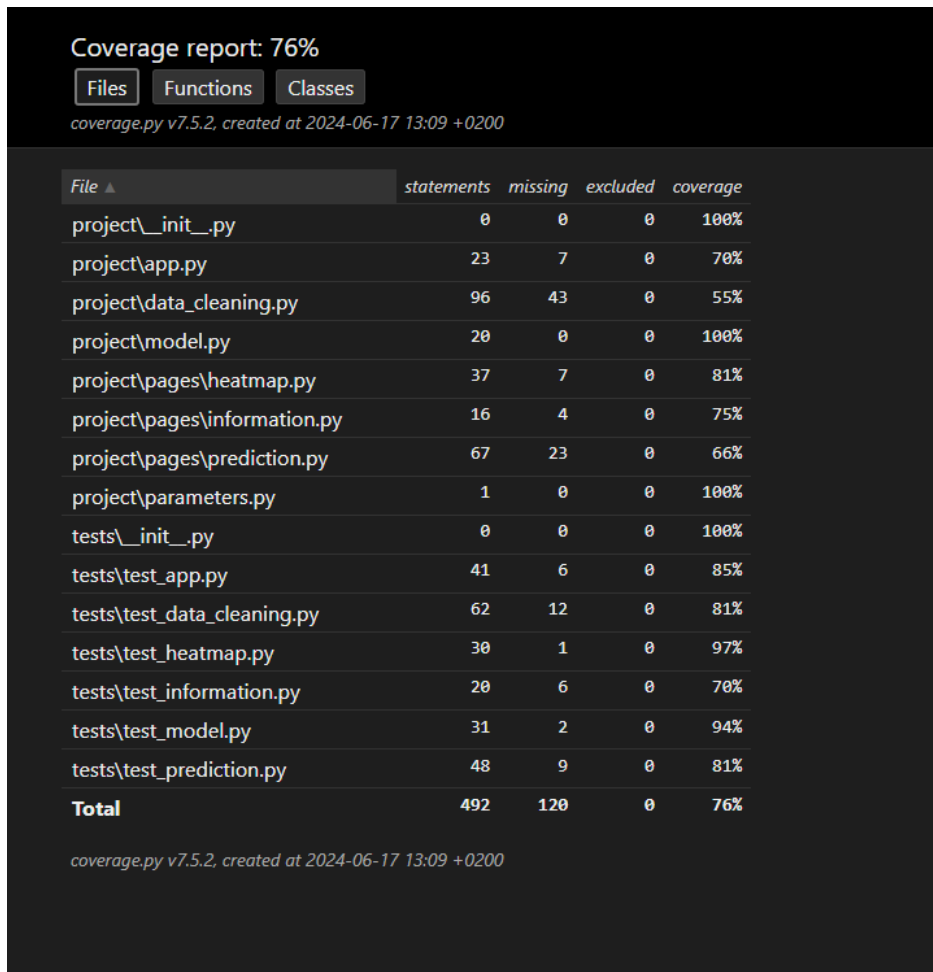


Figure 6. Coverage report on files.

2. **Coverage Report on Functions:** We also examined the coverage report for individual functions to ensure comprehensive testing (see Figure 7).

Coverage report: 76%

Files Functions Classes

coverage.py v7.5.2, created at 2024-06-17 13:09 +0200

File	function	statements	missing	excluded	coverage
project__init__.py	(no function)	0	0	0	100%
project\app.py	main	15	6	0	60%
project\app.py	(no function)	8	1	0	88%
project\data_cleaning.py	load_data	12	0	0	100%
project\data_cleaning.py	clean_data	23	15	0	35%
project\data_cleaning.py	clean_data.classify_incident_severity	10	10	0	0%
project\data_cleaning.py	detect_outliers_iqr	18	4	0	78%
project\data_cleaning.py	plot_boxplots	14	7	0	50%
project\data_cleaning.py	load_and_clean_data	7	7	0	0%
project\data_cleaning.py	(no function)	12	0	0	100%
project\model.py	load_random_forest_model	14	0	0	100%
project\model.py	(no function)	6	0	0	100%
project\pages\heatmap.py	heatmap_page	25	6	0	76%
project\pages\heatmap.py	heatmap_page.plot_heatmap	4	0	0	100%
project\pages\heatmap.py	(no function)	8	1	0	88%
project\pages\information.py	information_page	10	3	0	70%
project\pages\information.py	(no function)	6	1	0	83%
project\pages\prediction.py	predict_incident_severity	10	10	0	0%
project\pages\prediction.py	prediction_page	28	12	0	57%
project\pages\prediction.py	prediction_page.user_input_features	16	0	0	100%
project\pages\prediction.py	(no function)	13	1	0	92%
project\parameters.py	(no function)	1	0	0	100%
tests__init__.py	(no function)	0	0	0	100%
tests\test_app.py	TestDashboard.setUp	2	0	0	100%
tests\test_app.py	TestDashboard.tearDown	1	0	0	100%
tests\test_app.py	TestDashboard.test_main_information_page	15	5	0	67%
tests\test_app.py	(no function)	23	1	0	96%
tests\test_data_cleaning.py	TestLoadData.test_load_data_success	10	1	0	90%
tests\test_data_cleaning.py	TestLoadData.test_load_data_failure	7	2	0	71%
tests\test_data_cleaning.py	TestCleanData.setUp	1	0	0	100%
tests\test_data_cleaning.py	TestCleanData.test_clean_data	4	3	0	25%
tests\test_data_cleaning.py	TestCleanData.test_detect_outliers_iqr	6	0	0	100%
tests\test_data_cleaning.py	TestCleanData.test_plot_boxplots	2	0	0	100%
tests\test_data_cleaning.py	TestLoadAndCleanData.test_load_and_clean_data	5	5	0	0%
tests\test_data_cleaning.py	(no function)	27	1	0	96%
tests\test_heatmap.py	TestHeatmapPage.test_heatmap_page	11	0	0	100%
tests\test_heatmap.py	(no function)	19	1	0	95%
tests\test_information.py	TestInformationPage.setUp	1	0	0	100%
tests\test_information.py	TestInformationPage.tearDown	1	1	0	0%
tests\test_information.py	TestInformationPage.test_information_page_loads	2	2	0	0%
tests\test_information.py	TestInformationPage.test_information_page_error_handling	2	2	0	0%
tests\test_information.py	(no function)	14	1	0	93%
tests\test_model.py	TestLoadRandomForestModel.test_load_random_forest_model_success	6	1	0	83%
tests\test_model.py	TestLoadRandomForestModel.test_load_random_forest_model_file_not_found	3	0	0	100%
tests\test_model.py	TestLoadRandomForestModel.test_load_random_forest_model_exception	3	0	0	100%
tests\test_model.py	(no function)	19	1	0	95%
tests\test_prediction.py	TestPredictionPage.setUp	2	0	0	100%
tests\test_prediction.py	TestPredictionPage.tearDown	1	0	0	100%
tests\test_prediction.py	TestPredictionPage.test_prediction_page_successful_prediction	15	9	0	40%
tests\test_prediction.py	TestPredictionPage.test_prediction_page_invalid_input	2	0	0	100%
tests\test_prediction.py	(no function)	28	0	0	100%
Total		492	120	0	76%

coverage.py v7.5.2, created at 2024-06-17 13:09 +0200

Figure 7. Coverage report on functions.

3. **Coverage Report on Classes:** Finally, we checked the coverage report for the classes to ensure that all class methods were adequately tested (see Figure 8).

Coverage report: 86%

Files Functions **Classes**

coverage.py v7.5.2, created at 2024-06-17 13:09 +0200

File ▲	class	statements	missing	excluded	coverage
project__init__.py	(no class)	0	0	0	100%
project\app.py	(no class)	8	1	0	88%
project\data_cleaning.py	(no class)	12	0	0	100%
project\model.py	(no class)	6	0	0	100%
project\pages\heatmap.py	(no class)	8	1	0	88%
project\pages\information.py	(no class)	6	1	0	83%
project\pages\prediction.py	(no class)	13	1	0	92%
project\parameters.py	(no class)	1	0	0	100%
tests__init__.py	(no class)	0	0	0	100%
tests\test_app.py	TestDashboard	18	5	0	72%
tests\test_app.py	(no class)	23	1	0	96%
tests\test_data_cleaning.py	TestLoadData	17	3	0	82%
tests\test_data_cleaning.py	TestCleanData	13	3	0	77%
tests\test_data_cleaning.py	TestLoadAndCleanData	5	5	0	0%
tests\test_data_cleaning.py	(no class)	27	1	0	96%
tests\test_heatmap.py	TestHeatmapPage	11	0	0	100%
tests\test_heatmap.py	(no class)	19	1	0	95%
tests\test_information.py	TestInformationPage	6	5	0	17%
tests\test_information.py	(no class)	14	1	0	93%
tests\test_model.py	TestLoadRandomForestModel	12	1	0	92%
tests\test_model.py	(no class)	19	1	0	95%
tests\test_prediction.py	TestPredictionPage	20	9	0	55%
tests\test_prediction.py	(no class)	28	0	0	100%
Total		286	40	0	86%

Figure 8. Coverage report on classes.

Evidence:

https://github.com/BredaUniversityADSAI/2023-24d-fai1-adsai-teamwork-t2/blob/059b6e07397000ce9f32b25eb4ebfae9fff4d4dd/ILO_7_Deployment/235874_Teun_Unit_Testing.html

Evidencing

To provide further insight and evidence of our work, the following resources are available:

- **Full Project Folder:** [Project folder](#)
- **Demo Video of the Dashboard:** [Demo Video](#)
- **Sphinx Documentation:** [Sphinx html ink](#)
 - Also, see figure 5
- **Server Address:** <http://194.171.191.226:3551/>

Production Ready application

As demonstrated in the Dashboard's demo video, our application is user-friendly and easy to navigate. It features a clear and visible sidebar where users can select different pages. The homepage is set to the information page, so users immediately understand the tool's purpose and how to use it. The prediction page loads extremely fast, which is crucial for quickly determining the severity of road incidents. The heatmap page displays a heatmap of incidents in Breda, allowing users to easily identify hotspots for issues such as speeding. By knowing where these hotspots are, users can implement measures like speed bumps to reduce speeding incidents in high-risk areas.